```
Feature: Calculating earned Frequent Flyer points

Scenario: Earning standard domestic Frequent Flyer points
Given the flying distance between Sydney and Melbourne is 878 km
And I am a standard Frequent Flyer member
When I fly from Sydney to Melbourne
Then I should earn 439 travel points
And I should earn 40 status points
```

The code might call a web service on the server or do the calculations locally. From the point of view of the requirements, this doesn't matter. What's important here is the application's behavior from the point of view of the user.

If you place this scenario in a feature file called earning-frequent-flyer-points.feature in the feature directory, Cucumber-JS will be able to find it. You can run Cucumber-JS from the command line by typing `cucumber-js`. Like Cucumber-JVM, if there are no matching step implementations, Cucumber-JS will propose some skeleton steps to get you started:

```
$ cucumber-js
1 scenario (1 undefined)
5 steps (5 undefined)
```

You can implement step definitions for undefined steps with these snippets:

```
this.Given(/^the flying distance between Sydney and Melbourne is (\d+) km$/,
     function(arg1, callback) {
  // express the regexp above with the code you wish you had
  callback.pending();
});

this.Given(/^I am a standard Frequent Flyer member$/, function(callback) {
  // express the regexp above with the code you wish you had
  callback.pending();
});
...
```

This shows that you've specified pending scenarios. Now you can add some test logic.

6.8.3 Implementing the steps

Let's look at how to implement steps in Cucumber-JS. By default, Cucumber-JS will look for the step definition file in the step_definitions directory. A very simple step implementation file might look like this:

```
var stepDefinitionWrapper = function () {
  var should = require('chai').should()

  var distance_travelled, traveller_status;

  var calculate_travel_points = function(status, distance) {
    return distance / 2;
  }

  this.Given(/^the distance between (.*) and (.*) is (\d+) km$/,
             function(start, end, distance, callback) {
```

Chai is a BDD assertion library for JavaScript.

Variables that will be shared between steps.

In a real application, you'd call the application code here.